

DHT Project Report

Video link: https://youtu.be/sM8aYG_Rf-4

System Design:

This project extends our peer-to-peer network with distributed storage capabilities using a Distributed Hash Table (DHT). The system consists of a bootstrap node and three peer nodes, implementing three phases: file storage, key-value storage, and DHT-based routing.

Phase 1 - File Storage:

Each peer node (node.py) includes a ./storage directory for file storage. The /upload endpoint accepts file uploads via HTTP POST using Flask's request.files, saving files with file.save(). The /download/<filename> endpoint serves files using Flask's send_from_directory(). Docker volumes (-v ./storage/node1:/app/storage) ensure files persist across container restarts.

Phase 2 - Key-Value Store:

Each node maintains an in-memory dictionary (kv_store = {}) for key-value storage. The /kv POST endpoint stores data, while /kv/<key> GET retrieves it. All operations use with kv_lock: (threading.Lock()) for thread safety, preventing race conditions between the discovery thread and Flask request handlers.

Phase 3 - DHT Routing:

The system implements consistent hashing using SHA-1. The hash_key() function converts keys to 64-bit integers: hash_obj = hashlib.sha1(key.encode('utf-8')) then int.from_bytes(hash_obj.digest()[:8], 'big'). Similarly, hash_node() hashes node URLs.

The get_responsible_node(key) function determines data ownership:

1. Hash the key to get its position
2. Hash all node URLs to get their positions
3. Sort nodes by hash value
4. Find first node whose hash >= key hash (clockwise on hash ring)
5. If no node >= key hash, wrap around to first node

When a node receives a storage request, it calculates the responsible node. If responsible_node != node_url, it forwards the request using requests.post(). The response includes "forwarded_from" to show the routing path.

Network Architecture:

The bootstrap node (bootstrap.py) on port 5000 maintains peer registry. Three peer nodes run on ports 5001-5003, each with independent storage. Docker Compose orchestrates all containers on a bridge network. The peer_discovery_loop() background thread handles registration and updates every 30 seconds.

Screenshots

Phase 1 - File Upload Success:

```
● (venv) PS C:\Users\melod\CECS-327\Project-4-DHT> echo "File 1 for Node 1" > file1.txt
● (venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe -F "file=@file1.txt" http://localhost:5001/upload
{
  "filename": "file1.txt",
  "node_id": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "status": "uploaded"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT>
○ (venv) PS C:\Users\melod\CECS-327\Project-4-DHT> echo "File 2 for Node 1" > file2.txt
● (venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe -F "file=@file2.txt" http://localhost:5001/upload
● {
  "filename": "file2.txt",
  "node_id": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "status": "uploaded"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT>
● (venv) PS C:\Users\melod\CECS-327\Project-4-DHT> echo "File 3 for Node 1" > file3.txt
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe -F "file=@file3.txt" http://localhost:5001/upload
● {
  "filename": "file3.txt",
  "node_id": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "status": "uploaded"
}
○ (venv) PS C:\Users\melod\CECS-327\Project-4-DHT> |
```

Phase 1 - File Download:

```
● (venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe http://localhost:5001/download/file1.txt --output -
File 1 for Node 1
○ (venv) PS C:\Users\melod\CECS-327\Project-4-DHT> |
```

Phase 1 - Files Stored on Node:

```
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe http://localhost:5001/files
{
  "count": 3,
  "files": [
    "file1.txt",
    "file2.txt",
    "file3.txt"
  ],
  "node_id": "ee03803c-5e5e-4330-b003-948509bc6c9c"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe http://localhost:5002/files
{
  "count": 4,
  "files": [
    "docA.txt",
    "docB.txt",
    "docC.txt",
    "docD.txt"
  ],
  "node_id": "0fe78fa4-0743-433a-a66e-3b8667444244"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe http://localhost:5003/files
{
  "count": 5,
  "files": [
    "report1.txt",
    "report2.txt",
    "report3.txt",
    "report4.txt",
    "report5.txt"
  ],
  "node_id": "c8d0e2ee-b314-412f-8386-9f345090a811"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT>
```

Phase 2 - Store Key-Value:

```
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe -X POST http://localhost:5001/kv -H "Content-Type: application/json" -d '{"key": "color", "value": "blue"}'
{
  "forwarded_from": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "key": "color",
  "node_id": "c8d0e2ee-b314-412f-8386-9f345090a811",
  "responsible_node": "http://node3:5000",
  "status": "stored",
  "value": "blue"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT>
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe -X POST http://localhost:5001/kv -H "Content-Type: application/json" -d '{"key": "temperature", "value": "72"}'
{
  "forwarded_from": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "key": "temperature",
  "node_id": "c8d0e2ee-b314-412f-8386-9f345090a811",
  "responsible_node": "http://node3:5000",
  "status": "stored",
  "value": "72"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT>
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe -X POST http://localhost:5001/kv -H "Content-Type: application/json" -d '{"key": "status", "value": "active"}'
{
  "forwarded_from": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "key": "status",
  "node_id": "c8d0e2ee-b314-412f-8386-9f345090a811",
  "responsible_node": "http://node3:5000",
  "status": "stored",
  "value": "active"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT>
```

Phase 2 - Retrieve Value:

```
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe http://localhost:5001/kv/color
● {
  "forwarded_from": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "key": "color",
  "node_id": "c8d0e2ee-b314-412f-8386-9f345090a811",
  "responsible_node": "http://node3:5000",
  "value": "blue"
}
● (venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe http://localhost:5001/kv/temperature
{
  "forwarded_from": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "key": "temperature",
  "node_id": "c8d0e2ee-b314-412f-8386-9f345090a811",
  "responsible_node": "http://node3:5000",
  "value": "72"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe http://localhost:5001/kv/status
● {
  "forwarded_from": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "key": "status",
  "node_id": "c8d0e2ee-b314-412f-8386-9f345090a811",
  "responsible_node": "http://node3:5000",
  "value": "active"
}
○ (venv) PS C:\Users\melod\CECS-327\Project-4-DHT>
```

Phase 3 - DHT Forwarding:

```
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe -X POST http://localhost:5001/kv -H "Content-Type: application/json" -d '{"key": "banana", "value": "yellow"}'
{"node_id": "c8d0e2ee-b314-412f-8386-9f345090a811",
 "responsible_node": "http://node3:5000",
 "status": "stored",
 "value": "yellow"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT>
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe -X POST http://localhost:5001/kv -H "Content-Type: application/json" -d '{"key": "cherry", "value": "red"}'
{
  "forwarded_from": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "key": "cherry",
  "node_id": "c8d0e2ee-b314-412f-8386-9f345090a811",
  "responsible_node": "http://node3:5000",
  "status": "stored",
  "value": "red"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT>
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe -X POST http://localhost:5001/kv -H "Content-Type: application/json" -d '{"key": "date", "value": "brown"}'
{
  "forwarded_from": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "key": "date",
  "node_id": "0fe78fa4-0743-433a-a66e-3b8667444244",
  "responsible_node": "http://node2:5000",
  "status": "stored",
  "value": "brown"
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT>
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe -X POST http://localhost:5001/kv -H "Content-Type: application/json" -d '{"key": "elderberry", "value": "purple"}'
{
  "forwarded_from": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "key": "elderberry",
  "node_id": "c8d0e2ee-b314-412f-8386-9f345090a811",
  "responsible_node": "http://node3:5000",
  "status": "stored",
  "value": "purple"
}
```

Phase 3 - Data Distribution:

```
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe http://localhost:5001/kv
{
  "count": 0,
  "node_id": "ee03803c-5e5e-4330-b003-948509bc6c9c",
  "store": {}
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe http://localhost:5002/kv
{
  "count": 2,
  "node_id": "0fe78fa4-0743-433a-a66e-3b8667444244",
  "store": {
    "apple": "red",
    "date": "brown"
  }
}
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe http://localhost:5003/kv
{
  "count": 6,
  "node_id": "c8d0e2ee-b314-412f-8386-9f345090a811",
  "store": {
    "banana": "yellow",
    "cherry": "red",
    "color": "blue",
    "elderberry": "purple",
    "status": "active",
    "temperature": "72"
  }
}
```

Phase 3 - DHT Debug:

```
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> curl.exe http://localhost:5001/kv/debug
{
  "local_keys": [],
  "nodes": [
    {
      "hash": 2505155047060900102,
      "is_me": true,
      "url": "http://node1:5000"
    },
    {
      "hash": 2055427346256019087,
      "is_me": false,
      "url": "http://node2:5000"
    },
    {
      "hash": 14647101180111560074,
      "is_me": false,
      "url": "http://node3:5000"
    }
  ],
  "total_nodes": 3
}
```

Docker Containers:

```
(venv) PS C:\Users\melod\CECS-327\Project-4-DHT> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
9004a314874f   project-4-dht-node2   "python node.py"        6 minutes ago Up 6 minutes  0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp   node2
1749a0a313f4   project-4-dht-node1   "python node.py"        6 minutes ago Up 6 minutes  0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp   node1
cee9f12159eb   project-4-dht-node3   "python node.py"        6 minutes ago Up 6 minutes  0.0.0.0:5003->5000/tcp, [::]:5003->5000/tcp   node3
5469a6cd94d2   project-4-dht-bootstrap "python bootstrap.py"    6 minutes ago Up 6 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   bootstrap
```

Challenges and Solutions:

- Request Forwarding: Implemented proper error handling with try/except for network failures and 5-second timeouts
- Thread Safety: Used threading.Lock() for both kv_store and peers to prevent race conditions
- Hash Ring Wrap-around: Implemented proper modulo logic to wrap to first node when key hash exceeds all node hashes

Individual Contributions:

Melody:

- Implemented Phase 1 (file upload/download endpoints with Docker volumes)
- Implemented Phase 2 (key-value store with thread-safe operations)
- Implemented Phase 3 (DHT routing with hash_key(), hash_node(), get_responsible_node())
- Added request forwarding logic with proper error handling

- Tested 3-node deployment and captured screenshots
- Recorded demonstration video showing all three phases

Andrea:

- Created Dockerfile and bootstrap.Dockerfile
- Configured docker-compose.yml with volume mappings for all nodes
- Updated generate_compose.py to include storage volumes
- Wrote README.md with compilation and testing instructions
- Prepared project documentation and report formatting

References:

Docker Documentation: <https://docs.docker.com/>

Flask Documentation: <https://flask.palletsprojects.com/>

Python Hashlib: <https://docs.python.org/3/library/hashlib.html>

Python Threading: <https://docs.python.org/3/library/threading.html>

CECS 327 Course Materials